

Engenharia de Software II

André L. D. Rossi

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências (FC) / Departamento de Computação (DCo)
Bauru, SP - Brasil

Projeto baseado em Padrões

Projeto baseado em Padrões

Todos nós já nos deparamos com um problema de projeto e, silenciosamente, pensamos: *será que alguém já desenvolveu uma solução para esse problema?*

A resposta é quase sempre *sim!*

O problema é:

- Encontrar a solução
- Garantir que, de fato, adapte-se ao problema em questão
- Entender as restrições que talvez limitem a maneira pela qual a solução é aplicada
- Traduzir a solução proposta para seu ambiente de projeto

Projeto baseado em Padrões

Mas e se a solução fosse codificada de alguma forma?

E se existisse uma maneira padronizada de descrever um problema (de tal forma que pudéssemos pesquisá-lo) e um método organizado para representar a solução para o problema?

Os problemas de software seriam codificados e descritos usando-se um modelo padronizado e seriam propostas soluções (com restrições) para eles.

Denominado **padrões de projeto**, esse método codificado para descrição de problemas e suas soluções permite que os profissionais da engenharia de software adquiram conhecimento de projeto para que ele seja reutilizado.

Projeto baseado em Padrões

O início da história dos **padrões** começa com um arquiteto, Christopher Alexander.

Alexander encontrou um conjunto de problemas recorrentes toda vez que um edifício era projetado. Ele caracterizou esses problemas recorrentes e suas soluções como **padrões**, descrevendo-os da seguinte maneira:

Cada padrão descreve um problema que ocorre repetidamente em nosso ambiente e então descreve o cerne de uma solução para aquele problema para podermos usá-la repetidamente um milhão de vezes sem jamais ter de fazer a mesma coisa duas vezes.

As ideias de Alexander foram traduzidas inicialmente para o mundo do software em livros como os de Gamma [1].

Hoje, existem dezenas de repositórios de padrões, e projetos baseados em padrões podem ser aplicados em diversos domínios de aplicação.

Padrões de Projeto

Um *padrão de projeto* pode ser caracterizado como “uma **regra de três partes** que expressa uma relação entre um **contexto**, um **problema** e uma **solução**”.

Para projeto de software, o *contexto* permite ao leitor compreender o ambiente em que o problema reside e qual solução poderia ser apropriada nesse ambiente.

Um conjunto de requisitos, incluindo limitações e restrições, atua como um *sistema de forças*¹ que influencia a maneira pela qual o problema pode ser interpretado em seu contexto e como a solução pode ser aplicada eficientemente.

¹Forças são as características do problema e os atributos de uma solução que restringem a maneira como o projeto pode ser desenvolvido.

A maioria dos problemas possui várias soluções, porém uma solução é eficaz somente se for apropriada no contexto do problema existente.

É o sistema de forças que faz com que um projetista escolha uma solução específica. O intuito é fornecer uma solução que melhor atenda ao sistema de forças, mesmo quando essas forças são contraditórias.

Por fim, toda solução tem consequências que poderiam ter um impacto sobre outros aspectos do software, e ela própria poderia fazer parte do sistema de forças para outros problemas a serem resolvidos no sistema mais amplo.

Coplien² caracteriza um padrão de projeto eficaz da seguinte maneira:

- *Ele soluciona um problema:* os padrões apreendem soluções, não apenas estratégias ou princípios abstratos.
- *Ele é um conceito comprovado:* os padrões apreendem soluções com um histórico, não teorias ou especulação.
- *Uma solução não é óbvia:* muitas técnicas para resolução de problemas (como paradigmas ou métodos de projeto de software) tentam obter soluções com base nos primeiros princípios. Os melhores padrões geram uma solução para um problema indiretamente – uma abordagem necessária para os problemas de projeto mais difíceis.

²Coplien, J., “Software Patterns”, 2005, disponível em <http://hillside.net/patterns/definition.html>.

- *Ele descreve uma relação:* os padrões não apenas descrevem módulos, como também estruturas e mecanismos de sistema mais profundos.
- *O padrão possui um componente humano significativo (minimizar a intervenção humana).* Todo software visa a atender o conforto humano ou a qualidade de vida; os melhores padrões apelam explicitamente à estética e à utilidade.

Um padrão de projeto evita que tenhamos de “reinventar a roda” ou, pior ainda, inventar uma “nova roda” que não será perfeitamente redonda; será muito pequena para o uso pretendido e muito estreita para o terreno onde irá rodar.

Os padrões de projeto, se usados de maneira eficiente, invariavelmente o tornarão um melhor projetista de software.

Tipos de Padrões

Tipos de Padrões

Uma das razões para os engenheiros de software se interessarem (e ficarem intrigados) por padrões de projeto é o fato de os seres humanos serem inerentemente bons no reconhecimento de padrões.

No mundo real, os padrões que reconhecemos são aprendidos ao longo de toda uma vida de experiências.

Reconhecemos instantaneamente e compreendemos inerentemente seus significados e como eles poderiam ser usados.

Alguns desses padrões nos dão uma melhor visão do fenômeno da recorrência.

RubberNecking: olhar com curiosidade

O padrão **RubberNecking** produz resultados notavelmente previsíveis (um congestionamento), mas nada mais faz do que descrever um fenômeno.

No jargão dos padrões, ele poderia ser denominado **padrão não generativo**, pois descreve um contexto e um problema, mas não fornece nenhuma solução explícita.

Tipos de Padrões

Quando são considerados padrões de projeto de software, faz-se um esforço para identificar e documentar padrões **generativos**.

Em um ambiente ideal, um conjunto de padrões de projeto generativos poderia ser usado para “gerar” uma aplicação ou sistema computacional cuja arquitetura permitisse que se **adaptasse à mudança**.

Algumas vezes chamada generatividade, “a aplicação sucessiva de vários padrões, cada um deles encapsulando seu próprio problema e forças, desdobra-se em uma solução mais ampla que emerge indiretamente como resultado das soluções menores”.

Tipos de Padrões

Os padrões de projeto abrangem um amplo espectro de abstração e aplicação.

Tipos de Padrões

Os **padrões de arquitetura** descrevem problemas de projeto de caráter amplo e diverso, resolvidos usando-se uma abordagem estrutural.

Os **padrões de dados** descrevem problemas orientados a dados recorrentes e as soluções de modelagem de dados que podem ser usadas para resolvê-los.

Tipos de Padrões

Os **padrões de componentes** (também conhecidos como padrões de projeto) tratam de problemas associados ao desenvolvimento de subsistemas e componentes, da maneira pela qual eles se comunicam entre si e de seu posicionamento em uma arquitetura maior.

Os **padrões de interfaces** descrevem problemas comuns de interfaces do usuário e suas soluções, com um sistema de forças que inclui as características específicas dos usuários.

Tipos de Padrões

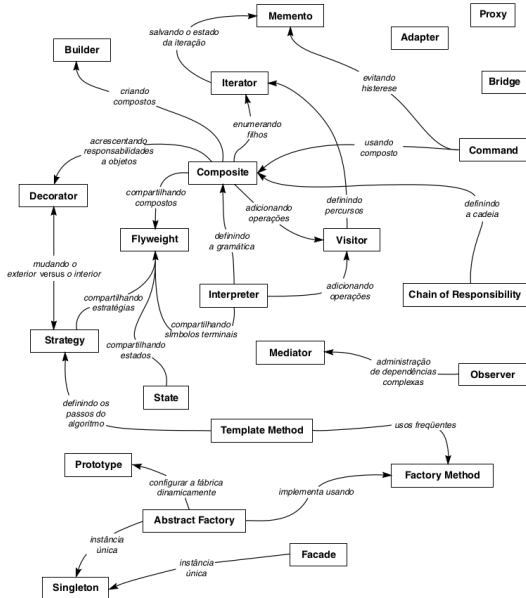
Os padrões para WebApp tratam de um conjunto de problemas encontrados ao se construir WebApps.

Em geral, incorporam muitas das demais categorias de padrões mencionados.

Em seu livro seminal sobre padrões de projeto, Gamma [1] e seus colegas descrevem três tipos de padrões particularmente relevantes para projetos orientados a objetos:

- padrões criacionais;
- padrões estruturais; e
- padrões comportamentais.

Relacionamento entre padrões de projeto



Padrões Criacionais

Padrões Criacionais

Os padrões criacionais se concentram na **criação, composição e representação** de objetos e dispõem de mecanismos que facilitam a instanciação de objetos.

Os padrões criacionais também impõem **restrições** sobre o **tipo** e **número** de objetos que podem ser criados em um sistema.

Exemplos de padrões criacionais:

- Singleton
- Método fábrica
- Fábrica abstrata
- Construtor

Singleton

Suponha uma classe *Logger*, usada para registrar as operações realizadas em um sistema:

```
void f() {
    Logger log = new Logger();
    log.println("Executando f");
    ...
}
void g() {
    Logger log = new Logger();
    log.println("Executando g");
    ...
}
void h() {
    Logger log = new Logger();
    log.println("Executando h");
    ...
}
```

Singleton

Problema:

Cada método cria sua própria instância de `Logger`. Isso pode não ser adequado para acesso a algum recurso (banco de dados, arquivos, etc).

Solução:

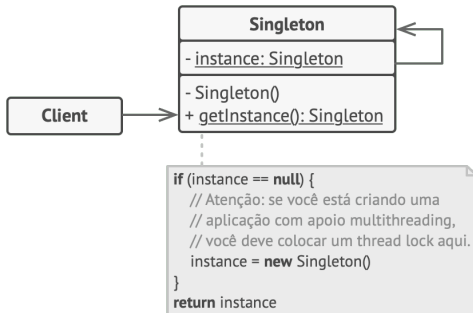


Figura 1: Padrão Criacional Singleton

(<https://refactoring.guru/pt-br/design-patterns/singleton>).

Singleton

```
class Logger {  
    // proibe clientes de chamar new  
    Logger()  
  
    private Logger() {}  
    // instancia unica  
    private static Logger instance;  
    public static Logger getInstance()  
    {  
        if (instance == null)  
            instance = new Logger();  
        return instance;  
    }  
    public void println(String msg)  
    {  
        // Poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```

```
void f() {  
    Logger log = Logger.getInstance();  
    log.println("Executando f");  
    ...  
}  
  
void g() {  
    Logger log = Logger.getInstance();  
    log.println("Executando g");  
    ...  
}  
  
void h() {  
    Logger log = Logger.getInstance();  
    log.println("Executando h");  
    ...  
}
```

Fábrica (estático)

Suponha um sistema distribuído baseado em TCP/IP. Nesse sistema, três funções f , g e h criam objetos do tipo *TCPChannel* para comunicação remota:

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

```
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

```
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

E se o sistema precisar usar UDP ao invés do TCP para comunicação?
Esse trecho de código não atende ao princípio Aberto/Fechado!

Fábrica (estático)

```
void f() {  
    TCPChannel c = new  
        TCPChannel();  
    ...  
}
```

```
void g() {  
    TCPChannel c = new  
        TCPChannel();  
    ...  
}
```

```
void h() {  
    TCPChannel c = new  
        TCPChannel();  
    ...  
}
```

```
class ChannelFactory {  
    // metodo fabrica estatico  
    public static Channel create() {  
        return new TCPChannel();  
    }  
}
```

```
void f() {  
    Channel c=ChannelFactory.create();  
    ...  
}
```

```
void g() {  
    Channel c=ChannelFactory.create();  
    ...  
}  
void h() {  
    Channel c=ChannelFactory.create();  
    ...  
}
```

Método Fábrica

Imagine agora que você poder sobrescrever o método fábrica em uma subclasse e mudar o produto que está sendo criado.

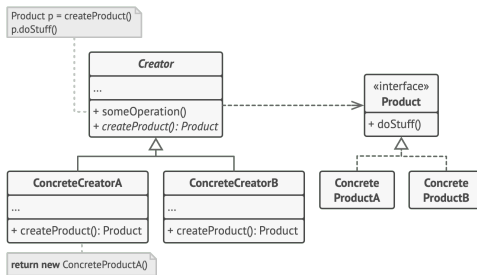


Figura 2: Uma fábrica é criada para cada produto.

Método Fábrica

Para o caso da “fábrica de canais de comunicação”:

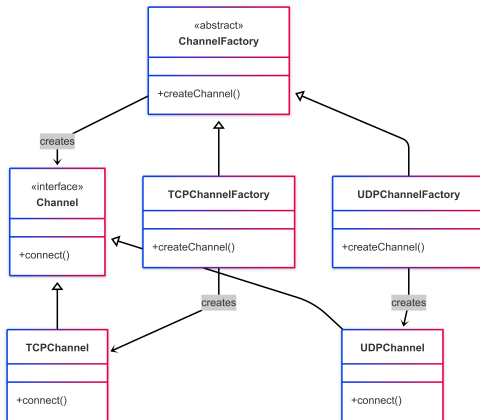


Figura 3: Padrão Criacional Método Fábrica para diferentes produtos (TCP e UDP).